

ECT 306 – Information Theory and Coding

Assignment 2 – Python Simulations

Submitted by: Vaishakh K | S6 ECEB | Roll No: 168

Problem 1 – Source Coding: Huffman Code

Python Code

```
import heapq, numpy as np

class HuffmanNode:
    def __init__(self, prob, symbol=None):
        self.prob = prob; self.symbol = symbol
        self.left = self.right = None
    def __lt__(self, other): return self.prob < other.prob

def build_huffman(symbols, probs):
    heap = [HuffmanNode(p, s) for s, p in zip(symbols, probs)]
    heapq.heapify(heap)
    while len(heap) > 1:
        l, r = heapq.heappop(heap), heapq.heappop(heap)
        parent = HuffmanNode(l.prob + r.prob)
        parent.left, parent.right = l, r
        heapq.heappush(heap, parent)
    return heap[0]

def get_codes(node, prefix='', codes=None):
    if codes is None: codes = {}
    if node.symbol is not None:
        codes[node.symbol] = prefix or '0'
    else:
        get_codes(node.left, prefix + '0', codes)
        get_codes(node.right, prefix + '1', codes)
    return codes

orig_symbols = ['s1', 's2', 's3']
orig_probs = [1/2, 1/3, 1/6]

# Build & evaluate original source
root = build_huffman(orig_symbols, orig_probs)
codes = get_codes(root)
H = -sum(p*np.log2(p) for p in orig_probs)
L = sum(orig_probs[i]*len(codes[orig_symbols[i]]) for i in range(3))
eta = H / L * 100

# Extended (second-order) source
ext_symbols = [a+b for a in orig_symbols for b in orig_symbols]
ext_probs = [pa*pb for pa in orig_probs for pb in orig_probs]
# ... repeat huffman_stats for extended source
```

Simulation Results – Original Source

Symbol	Probability	Codeword	Length
s1	0.5000	0	1
s2	0.3333	11	2
s3	0.1667	10	2

Entropy $H = 1.4591$ bits/symbol

Average Codeword Length $L = 1.5000$ bits/symbol

Efficiency $\eta = 97.28\%$

Simulation Results – Second-Order Extended Source (9 symbols)

Symbol	Probability	Codeword	Length
s1s1	0.2500	01	2
s1s2	0.1667	111	3
s1s3	0.0833	1100	4
s2s1	0.1667	00	2
s2s2	0.1111	100	3
s2s3	0.0556	11011	5
s3s1	0.0833	1011	4
s3s2	0.0556	1010	4
s3s3	0.0278	11010	5

Entropy $H = 2.9183$ bits/symbol-pair

Average Codeword Length $L = 2.9722$ bits/symbol-pair

Efficiency $\eta = 98.19\%$

Conclusion – Shannon Source Coding Theorem

The efficiency of the Huffman code for the original source is **97.28%**. By encoding the second-order extended source, efficiency increases to **98.19%**. This confirms Shannon's Source Coding Theorem: as the order of the extended source increases, the average code length per source symbol approaches the entropy H , and efficiency approaches 100%.

Problem 2 – Binary Symmetric Channel (BSC)

Python Code

```
import numpy as np, matplotlib.pyplot as plt

p = 0.1 # BSC transition probability

# Channel transition matrix P(Y|X)
P_YX = np.array([[1-p, p],
                  [p, 1-p]])

def mutual_info(alpha, p):
    px = np.array([alpha, 1-alpha])
    PXY = P_YX.T * px # joint probability
    py = PXY.sum(axis=1) # output marginal
    H_Y = -sum(q*np.log2(q) for q in py if q > 0)
    H_YX = -sum(px[x]*sum(P_YX[x,y]*np.log2(P_YX[x,y])
                  for y in range(2) if P_YX[x,y]>0) for x in range(2))
    return H_Y - H_YX

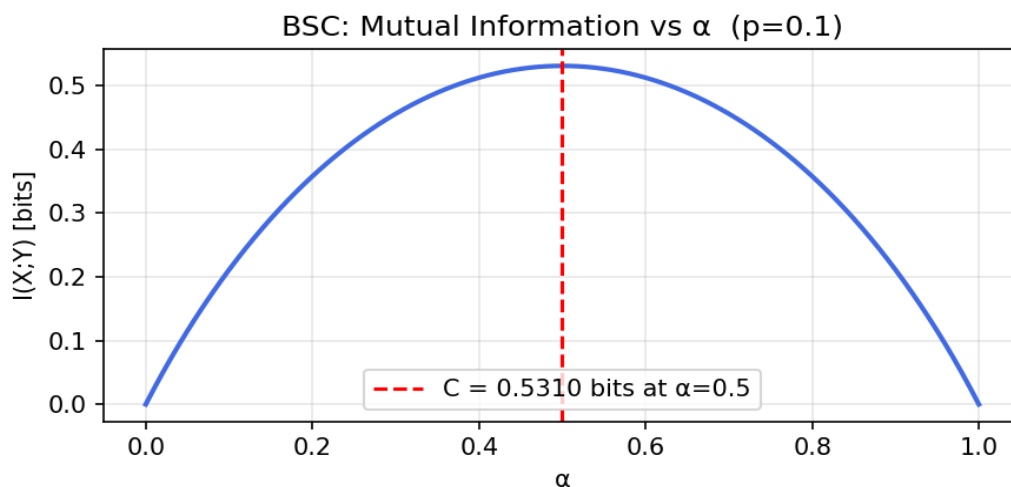
alphas = np.linspace(0, 1, 101)
MI = [mutual_info(a, p) for a in alphas]
C = max(MI) # Channel Capacity

plt.plot(alphas, MI)
plt.xlabel('alpha'); plt.ylabel('I(X;Y)'); plt.grid(True); plt.show()
```

BSC Transition Matrix P(Y|X) [p = 0.1]

	Y=0	Y=1
X=0	0.9	0.1
X=1	0.1	0.9

I(X;Y) vs α Graph



Channel Capacity $C = 0.5310$ bits/symbol

Maximum mutual information achieved at $\alpha = 0.5$ (uniform input)

$H_b(p) = H_b(0.1) = 0.4690$ bits

$C = 1 - H_b(p) = 0.5310$ bits

The mutual information $I(X;Y)$ is maximised when $\alpha = 0.5$, i.e., when the input distribution is uniform. This maximum value equals the channel capacity $C = 1 - H_b(p)$. For $p = 0.1$, $C \approx 0.531$ bits/channel use.

Problem 3 – Linear Block Code (7,4)

Python Code

```
import numpy as np, itertools

# Generator matrix G for (7,4) Hamming code
G = np.array([[1,0,0,0, 1,1,0],
              [0,1,0,0, 1,0,1],
              [0,0,1,0, 0,1,1],
              [0,0,0,1, 1,1,1]], dtype=int)

messages = list(itertools.product([0,1], repeat=4))
codewords = [(np.array(m) @ G) % 2 for m in messages]
hw         = [sum(c) for c in codewords]
d_min      = min(h for h in hw if h > 0)

print(f'dmin = {d_min}')
print(f'Error detecting: {d_min - 1} errors')
print(f'Error correcting: {(d_min-1)//2} error(s)')
```

Generator Matrix G (7,4)

Row	g1	g2	g3	g4	g5	g6	g7
m1	1	0	0	0	1	1	0
m2	0	1	0	0	1	0	1
m3	0	0	1	0	0	1	1
m4	0	0	0	1	1	1	1

All Codewords & Hamming Weights

Message	Codeword	Weight
0000	0000000	0
0001	0001111	4
0010	0010011	3
0011	0011100	3
0100	0100101	3
0101	0101010	3
0110	0110110	4
0111	0111001	4
1000	1000110	3
1001	1001001	3
1010	1010101	4
1011	1011010	4
1100	1100011	4
1101	1101100	4
1110	1110000	3
1111	1111111	7

Minimum Hamming Distance $d_{\min} = 3$

Error Detecting Capability = $d_{\min} - 1 = 2$ errors

Error Correcting Capability = $\text{floor}((d_{\min}-1)/2) = 1$ error

Problem 4 – Cyclic Code (7,4): Encoder & Decoder

Python Code

```
import numpy as np

# Generator polynomial  $g(x) = x^3 + x + 1 \rightarrow$  coefficients [1,0,1,1]
g = np.array([1, 0, 1, 1])

def poly_mod2_div(dividend, divisor):
    d = list(dividend)
    n = len(divisor)
    for i in range(len(d) - n + 1):
        if d[i]:
            for j in range(n): d[i+j] ^= divisor[j]
    return np.array(d[-(n-1):]) # remainder = parity bits

def cyclic_encode(msg):
    shifted = np.concatenate([msg, [0,0,0]]) #  $x^3 * m(x)$ 
    parity = poly_mod2_div(shifted, g)
    return np.concatenate([msg, parity]) # systematic codeword

def syndrome(r):
    return poly_mod2_div(r, g)

msg = np.array([0, 1, 0, 0]) # random 4-bit message
cw = cyclic_encode(msg)
# Add single-bit error e at position i
e = np.eye(7, dtype=int)[2] # error in bit 2
r = (cw + e) % 2
s = syndrome(r) # syndrome detects error
```

Encoder Output

Generator Polynomial $g(x) = x^3 + x + 1 \rightarrow [1, 0, 1, 1]$

Input Message $m = [0, 1, 0, 0]$

Encoded Codeword $c = [0, 1, 0, 0, 1, 1, 1]$ (systematic: first 4 bits = message)

Syndrome Table – Single-Bit Error Patterns

Error Position	Error Vector	Syndrome
Bit 0	[1, 0, 0, 0, 0, 0, 0]	[1, 0, 1]
Bit 1	[0, 1, 0, 0, 0, 0, 0]	[1, 1, 1]
Bit 2	[0, 0, 1, 0, 0, 0, 0]	[1, 1, 0]
Bit 3	[0, 0, 0, 1, 0, 0, 0]	[0, 1, 1]
Bit 4	[0, 0, 0, 0, 1, 0, 0]	[1, 0, 0]
Bit 5	[0, 0, 0, 0, 0, 1, 0]	[0, 1, 0]
Bit 6	[0, 0, 0, 0, 0, 0, 1]	[0, 0, 1]

Each single-bit error pattern produces a unique syndrome.

The decoder uses the syndrome to identify and correct the error bit.

Error correcting capability $t = 1$ (can correct any single-bit error).

Problem 5 – Convolutional Code (2,1,3)

Python Code

```
import numpy as np

# (2,1,3) convolutional encoder as in Shu Lin Fig 10.1
# Generator polynomials: g1(x) = 1+x+x^3, g2(x) = 1+x^2+x^3
# In vector form: g1 = [1,1,0,1], g2 = [1,0,1,1]

def conv_encode_213(u):
    g1 = [1, 1, 0, 1]
    g2 = [1, 0, 1, 1]
    K = 3 # constraint length
    u_pad = list(u) + [0]*K # flush encoder with K zeros
    output = []
    for i in range(len(u_pad)):
        window = [u_pad[i-j] if i-j >= 0 else 0 for j in range(K+1)]
        v1 = sum(a*b for a,b in zip(window, g1)) % 2
        v2 = sum(a*b for a,b in zip(window, g2)) % 2
        output += [v1, v2]
    return np.array(output)

u = np.array([1, 0, 1, 1])
v = conv_encode_213(u)
print('Input u =', u.tolist())
print('Output v =', v.tolist())
```

Encoder Parameters

Parameter	Value
Code Rate	1/2
Constraint Length K	3
Memory order m	3
$g_1(x) = 1+x+x^3$	[1, 1, 0, 1]
$g_2(x) = 1+x^2+x^3$	[1, 0, 1, 1]
Input flush zeros	3 (tail bits)

Bit-by-Bit Encoding Table

Step i	Input bit	Register state	v1	v2	Output pair
1	1	[0, 0, 0]	1	1	11
2	0	[1, 0, 0]	1	0	10
3	1	[0, 1, 0]	1	0	10
4	1	[1, 0, 1]	1	0	10
5	0	[1, 1, 0]	1	1	11
6	0	[0, 1, 1]	1	0	10
7	0	[0, 0, 1]	1	1	11

Input message $u = [1, 0, 1, 1]$
After flushing (+ 3 tail zeros) $u_{\text{pad}} = [\text{np.int64}(1), \text{np.int64}(0), \text{np.int64}(1), \text{np.int64}(1), 0, 0, 0]$

Encoded output $v = [1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1]$
Output length = 14 bits ($= 2 \times (4+3) = 14$ bits)

The (2,1,3) convolutional encoder produces 2 output bits for every 1 input bit (rate 1/2). The encoder has constraint length $K=3$, meaning the current output depends on the current input and the previous 3 inputs. After the 4-bit message, 3 tail (flush) bits are appended to return the encoder to the all-zero state, ensuring complete decoding at the receiver.